

Sensing-Project 全栈化重构平台 — 技术文档

版本: v1.0 | 日期: 2026-05-01 | 适用于 sqc v0.1.0

目录

- 1. 平台概览
- 2. 物理背景
- 3. 全栈架构
- 4. 模块详解
- 5. 数据流
- 6. 快速开始
- 7. 扩展指南
- 8. API 参考
- 9. 测试与回归
- 10. 设计原则与约定

1. 平台概览

1.1 项目定位

Sensing-Project 是一个**研究型量子传感仿真框架**，以 flux-tunable Transmon 超导量子比特为物理对象，研究**时变磁通信号的传感、重建、标定与反演**。

平台将超导量子计算机的控制栈从"单文件脚本"升级为**六层全栈架构**，每一层对应真实 cQED（circuit QED）实验中的物理组件或控制逻辑。

1.2 三大科研主线

#	主线	物理目标	sqc/ 归属
1	波形重建	从 Ramsey / Cryoscope / 瞬态测量恢复片上 $\Phi(t)$	sqc/reconstruction/
2	qubit 标定	标定 f_{01} , $f(\Phi)$, 灵敏度 κ	sqc/calibration/
3	波形预失真	建模 AWG→芯片传递函数，设计补偿滤波器	sqc/hardware/ + sqc/calibration/

1.3 关键特性

- **纯 Python + QuTiP**: 基于 QuTiP 的 `mesolve()` 进行全密度矩阵时间演化
- **自然单位制**: $\hbar=1$ ，频率/能量单位 rad-GHz，时间单位 ns，磁通单位 Φ_0
- **不可变参数对象**: `QubitSpec` 使用 `@dataclass(frozen=True)`，杜绝"物理参数被实验状态污染"
- **无副作用 Hamiltonian 构造**: `HamiltonianBuilder.build()` 纯函数，输入 device + flux + pulse，输出 `H_list`
- **永久向后兼容**: `src/` 目录永久保留，`src_mirror/` 提供从 `sqc/` 重导出的 facade

2. 物理背景

2.1 Transmon 量子比特

Transmon 是一种基于约瑟夫森结的超导量子比特，其哈密顿量为（Koch 2007）：

$$H = 4 \cdot EC \cdot n^2 - EJ(\Phi) \cdot \cos(\phi)$$

其中：

- **EC** = $e^2/(2C\Sigma)$ ：充电能（单电子库仑能）
- **EJ(Φ)** = $EJ_0 \cdot |\cos(\pi \cdot \Phi/\Phi_0)|$ ：有效约瑟夫森能（SQUID 调制）
- **n, ϕ**：Cooper 对数和超导相位算符

在 $EJ/EC \gg 1$ 的 Transmon 区域，哈密顿量可展开为 Duffing 振子形式：

$$H \approx \omega_T \cdot (n + \frac{1}{2}) + (\alpha/2) \cdot (n^2 - n)$$

- **ω_T(Φ)** = $\sqrt{8 \cdot EJ(\Phi) \cdot EC} - EC$ ：|0⟩→|1⟩ 跃迁频率（rad·GHz）
- **α** = $-EC$ ：非谐性（anharmonicity），保证二能级子空间可寻址

2.2 磁通传感原理

外部磁通 Φ(t) 改变 EJ(Φ)，进而改变 ω_T(Φ)。通过 Ramsey 干涉测量 qubit 的相干相位累积：

$$\phi(\tau) = \int_0^\tau \Delta\omega(t') \, dt' = \int_0^\tau \kappa \cdot \Phi(t') \, dt'$$

其中 **κ = dω_T/dΦ** 是频率-磁通灵敏度（flux sensitivity）。

2.3 参数推荐范围（Gao 2021 §III.B）

参数	推荐范围	本项目默认值
EJ/h	10–25 GHz	15 GHz
EC/h	160–400 MHz	200 MHz
EJ/EC	~50	~75
f ₀₁	4–8 GHz	~4.7 GHz
α/h	200–300 MHz	200 MHz

3. 全栈架构

3.1 六层模型（Gao 2021 Fig.1(a)）

workflows/ calibration	顶层科研流程	← 串联 experiment + reconstruction +
calibration/	标定 workflow	← 产出 CalibrationTable
reconstruction/	波形重建算法	← 消费测量数据，产出重建波形
experiments/ readout + runner	实验协议对象	← 组合 device + flux + sequence +
simulation/ noise	QuTiP 调用层	← HamiltonianBuilder, MesolveRunner,
control/	控制脉冲层	← Waveform, FluxSignal, Pulse, gates
hardware/ TransferMatrix	控制电子学/链路层	← Controlline, DistortionModel,
devices/	物理器件层	← QubitSpec, TransmonQubit, Resonator,
ChipTopology		

依赖规则：上层可依赖下层，禁止反向依赖。例如 `reconstruction` 不可 import `workflows`。

3.2 目录树

sqc/	
├─ devices/	物理器件 (Transmon, Resonator, Coupler, ChipTopology)
├─ hardware/	控制电子学 (Controlline, DistortionModel, TransferMatrix, Readout)
├─ control/	控制脉冲 (Waveform, FluxSignal, Pulse, Sequence, Gates)
├─ simulation/	QuTiP 接口 (HamiltonianBuilder, MesolveRunner, noise, result)
├─ experiments/	实验协议 (Rabi, Ramsey, Echo, Transient, Cryoscope)
├─ calibration/	标定流程 (QubitFrequency, FluxResponse, TransferFunction, Predistortion)
├─ reconstruction/	波形重建 (Wiener, Hammerstein, LM, Cryoscope, Kernel, basis)
└─ workflows/	顶层流程 (WaveformReconstruction, QubitCalibration, PredistortionValidation, ZCrosstalk)

4. 模块详解

4.1 devices/ — 物理器件层

QubitSpec (sqc/devices/transmon.py)

不可变 (frozen dataclass) 的 Transmon 参数描述对象，不存储任何实验状态。

```

from sqc.devices.transmon import QubitSpec
import numpy as np

spec = QubitSpec(
    name="Q0",
    EC=2 * np.pi * 0.2,    # rad·GHz
    EJ=2 * np.pi * 15,    # rad·GHz
    T1=10000.0,            # ns
    T2=8000.0,             # ns
    flux_bias=0.0,         #  $\Phi_0$ 
    n_levels=3,
)

f01 = spec.frequency()    #  $f_{01}$  at current flux
f01_at_flux = spec.frequency(0.1) #  $f_{01}$  at  $\Phi=0.1 \Phi_0$ 
kappa = spec.sensitivity() #  $d\omega/d\Phi$  at current flux
new_spec = spec.with_flux(0.05) # return NEW QubitSpec, does NOT mutate

```

关键方法：

- `frequency(flux=None)` $\rightarrow f_{01}(\Phi)$
- `anharmonicity()` $\rightarrow \alpha = -EC$
- `sensitivity(flux=None, delta=1e-6)` $\rightarrow \kappa = df_{01}/d\Phi$
- `with_flux(new_flux)` $\rightarrow$ 返回新对象（不可变模式）
- `EJ_at(flux=None)` $\rightarrow EJ(\Phi) = EJ_0 |\cos(\pi\Phi)|$

TransmonQubit (`sqc/devices/transmon.py`)

向后兼容的完整 qubit 类，内部包装 `QubitSpec`。保留所有 `src/qubit.py` 的方法签名。

```

from sqc.devices.transmon import TransmonQubit

q = TransmonQubit(EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=10000, T2=8000)
q.qubit_in_mag(flux_signal)    # 计算 H_list (副作用: 设置 q.H_list)
H = q.get_hamiltonian()        # 静态哈密顿量
H_rwa = q.get_hamiltonian_rwa(omega_d) # 旋转波近似哈密顿量
c_ops = q.get_collapse_operators() # Lindblad 坍缩算符

```

Resonator (`sqc/devices/resonator.py`)

多模 LC 谐振腔模型。替代旧 `Cavity` 类 (`src/qubit.py`)。

ChipTopology (`sqc/devices/chip.py`)

多 qubit 芯片拓扑容器。管理 qubit 列表、谐振腔列表、耦合映射、控制线分配。

```
from sqc.devices.chip import ChipTopology

chip = ChipTopology(
    qubits=[q0, q1],
    resonators=[resonator],
    couplings={"Q0", "R0": 0.05},
)
op_lifted = chip.lift_qubit_op(sigma_z, "Q0") # 嵌入完整 Hilbert 空间
```

4.2 hardware/ — 控制电子学层

ControlLine (sqc/hardware/control_line.py)

一条物理控制线的完整描述。

```
from sqc.hardware.control_line import ControlLine

line = ControlLine(
    name="Z0", kind="z", source="AWG0:CH1", target="Q0",
    transfer_function=SingleExponentialDistortion(amplitude=0.01, tau=50.0),
)
on_chip = line.apply(awg_waveform) # AWG → 片上（前向失真）
awg = line.predistort(target, designer) # 片上目标 → AWG 波形（逆滤波）
```

DistortionModel (sqc/hardware/distortion.py)

五种失真模型：

模型	描述	参数
SingleExponentialDistortion	单指数尾巴（最常用）	amplitude, tau
MultiExponentialDistortion	多时间常数尾巴	amplitudes[], taus[]
FIRDistortion	有限冲激响应滤波器	taps[]
IIRDistortion	无限冲激响应滤波器	b[], a[]
CustomTransferDistortion	自定义频域 $H(\omega)$	omega_grid, H_grid

所有模型实现四个方法：

- `apply(waveform, dt)` → 离散时间滤波输出
- `step_response(t)` → 阶跃响应 $s(t)$
- `impulse_response(t)` → 冲激响应 $h(t)$
- `frequency_response(omega)` → 复频域响应 $H(\omega)$
- `apply_to_waveform(wf)` → Waveform 对象便捷接口

TransferMatrix (sqc/hardware/transfer_matrix.py)

多 qubit Z 线串扰矩阵: $\Phi_j(\omega) = \sum_i H_{ji}(\omega) \cdot V_i(\omega)$ 。

```
from sqc.hardware.transfer_matrix import TransferMatrix

tm = TransferMatrix.from_dc_matrix(
    dc_matrix=np.array([[1.0, 0.05], [0.03, 1.0]]),
    source_names=["QA", "QB"],
    target_names=["QA", "QB"],
)
fluxes = tm.apply({"QA": pulse_on_A, "QB": pulse_on_B})
phi_B = fluxes["QB"] # includes crosstalk from QA
```

ReadoutModel (sqc/hardware/readout.py)

- **IdealProjectiveReadout**: 投影测量到 $|1\rangle$
- **IQReadoutModel**: IQ 解调读出 (两路 Ramsey, $\pi/2$ 相位差)

4.3 control/ — 控制脉冲层

Waveform (sqc/control/waveform.py)

通用时域波形 (无物理语义)。

```
from sqc.control.waveform import Waveform

wf = Waveform(
    t_list=np.linspace(0, 100, 1000),
    samples=np.sin(2*np.pi*0.01*t),
)
wf.value_at(25.0) # 采样保持插值
wf.truncate(20, 80) # 返回新对象, 边缘置零
wf.plot() # matplotlib 可视化
```

FluxSignal (sqc/control/flux_signal.py)

磁通信号 (继承 Waveform, samples 单位为 Φ_0)。兼容旧 **Signal**(type=N, ...) 构造方式。

```
from sqc.control.flux_signal import FluxSignal

# type-based 构造 (兼容旧 API)
phi = FluxSignal(type=2, t_list=t, amplitude=0.001, frequency=0.01) # 正弦
phi = FluxSignal(type=3, t_list=t, amplitude=0.01, center=50, width=3) # 高斯
phi = FluxSignal(type=8, t_list=t, signal=custom_array) # 自定义
```

类型映射 (旧 type $\rightarrow$ 新 kind):

type	kind	描述
0	"zero"	零信号
1	"constant"	常数
2	"sinusoidal"	正弦波
3	"gaussian"	高斯脉冲
4	"asymmetric_pulse"	非对称双指数
5	"double_peak"	双峰
6	"basis_expansion"	基函数展开
7	"wavepacket"	波包
8	"custom"	自定义数组

Pulse / CompositePulse (sqc/control/pulse.py)

微波控制脉冲。支持 lab frame / rotating frame + RWA。

```
from sqc.control.pulse import Pulse, CompositePulse

pulse = Pulse(
    Omega=Signal(type=3, t_list=t, amplitude=1.0, center=t_mid, width=sigma),
    frame=1, omega_d=qubit.frequency, phase=0, is_rwa=True,
)
H_pulse = pulse.hamiltonian # QuTiP list format
```

序列工厂 (sqc/control/sequence.py)

```
from sqc.control.sequence import (
    create_ramsey_pulse,      #  $\pi/2 - \tau - \pi/2$ 
    create_echo_pulse,       #  $\pi/2 - \tau - \pi - \tau - \pi/2$ 
    create_diff_echo_pulse,  #  $\pi/2 - [\text{echo}]^k - \pi/2$ 
    create_cpmg_pulse,       # CPMG 序列
    create_cryoscope_pulse,  #  $Y/2 - Z(t) - Y/2$ 
)
```

4.4 simulation/ — 仿真层

HamiltonianBuilder (sqc/simulation/hamiltonian.py)

纯函数：从 device + flux + pulse 构造 QuTiP H_list，不修改任何输入对象。

```

from sqc.simulation.hamiltonian import HamiltonianBuilder

H_list, t_global = HamiltonianBuilder.build(
    qubit=spec,                # QubitSpec 或 TransmonQubit
    flux_signal=phi,           # FluxSignal 或 None
    pulse=ramsey_pulse,        # PulseBase 或 None
    frame="rotating",
    omega_d=qubit.frequency,
)
# H_list 直接传给 QobjEvo 或 mesolve

```

MesolveRunner / SlidingMeasurementRunner (sqc/simulation/runner.py)

```

from sqc.simulation.runner import MesolveRunner, SlidingMeasurementRunner

runner = MesolveRunner()
result = runner.run(H_list, psi0, t_list, c_ops, e_ops)

sliding = SlidingMeasurementRunner()
result = sliding.run(qubit, flux_signal, control_pulse, scan_list)

```

ExperimentResult (sqc/simulation/result.py)

统一的实验结果容器，支持 pickle 序列化。

```

result.data["p_e"]          # 命名数据数组
result.axes["tau"]          # 命名轴数组
result.metadata             # qubit_spec, experiment_class, git_sha 等
result.save("output.pkl")
result = ExperimentResult.load("output.pkl")

```

4.5 experiments/ — 实验协议层

每个 Experiment 类 = device + flux_signal + sequence + readout + runner 的组合。

```

from sqc.experiments.ramsey import RamseyExperiment

exp = RamseyExperiment(
    qubit=q,
    flux_signal=phi,
    tau_list=np.linspace(0, 250, 500),
)
result = exp.run()
# result.data["p_e"]      → Ramsey 干涉条纹
# result.axes["tau"]      → 自由演化时间轴

```



```
from sqc.experiments.transient import TransientSensingExperiment
from sqc.experiments.echo import DiffEchoExperiment
from sqc.experiments.cryoscope import CryoscopeExperiment
from sqc.experiments.rabi import RabiExperiment
```

4.6 reconstruction/ — 波形重建层

所有重建类继承 `Reconstruction ABC`，实现 `reconstruct(measurement, kernel, calibration)`。

类	算法	输入 → 输出
<code>WienerReconstruction</code>	线性 Wiener 反卷积	$\Delta p + \text{kernel} \rightarrow \Phi(t)$
<code>HammersteinWienerReconstruction</code>	块结构非线性：Wiener + 色散反演	$\Delta p + \text{kernel} \rightarrow \Phi(t)$
<code>LMReconstruction</code>	Levenberg-Marquardt 全密度矩阵优化	$p_{\text{meas}} \rightarrow \Phi(t)$ (基函数参数化)
<code>RamseyIQReconstruction</code>	IQ 解调 $\rightarrow \arcsin \rightarrow B(\tau)$	$(I, Q) \rightarrow B(\tau)$
<code>RamseyUnwrapReconstruction</code>	相位解缠绕	$p_e(\tau) \rightarrow B(\tau)$
<code>DiffEchoReconstruction</code>	差分回波直接公式	$p_{e_list} \rightarrow B$
<code>CryoscopeReconstruction</code>	相位微分类	$\varphi(\text{trunc}) \rightarrow h(t)$

KernelEstimator (`sqc/reconstruction/kernel.py`)

统一的控制核函数估计器（消除了旧代码三处重复实现）。

```
from sqc.reconstruction.kernel import KernelEstimator

estimator = KernelEstimator(stim_amplitude=0.0215, stim_width=3.0)
t_samples, kernel = estimator.estimate(pulse, qubit)
```

4.7 calibration/ — 标定层

```
from sqc.calibration.qubit_frequency import QubitFrequencyCalibration
cal = QubitFrequencyCalibration(qubit=q)
table = cal.calibrate() # → CalibrationTable

from sqc.calibration.flux_response import FluxResponseCalibration
cal = FluxResponseCalibration(qubit=q, method="ramsey")
table = cal.calibrate()

from sqc.calibration.predistortion import PredistortionDesigner
designer = PredistortionDesigner(method="fir_inverse", n_taps=64)
```

```
inverse_model = designer.design(distortion_model)
predistorted = designer.predistort(target_waveform, distortion_model)
```

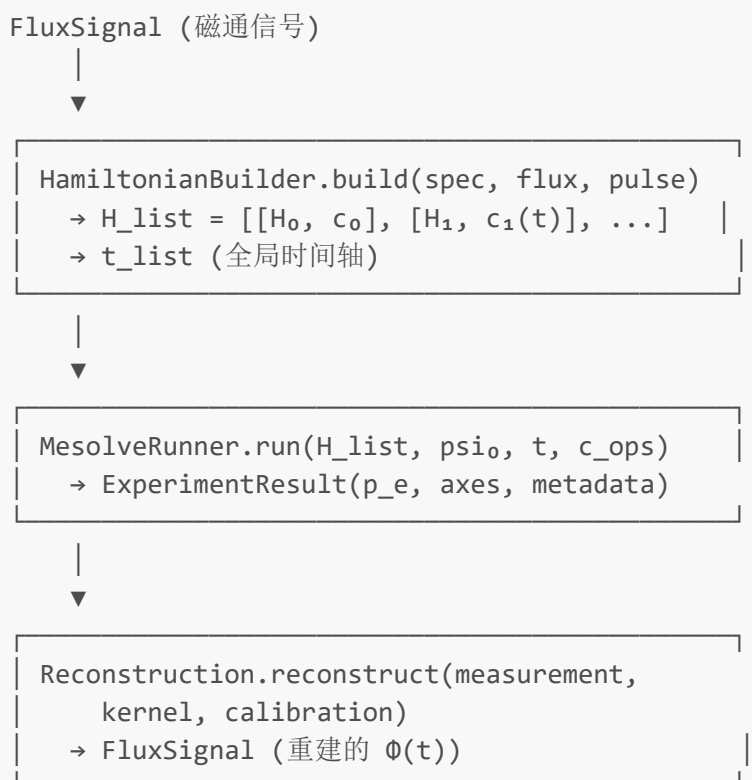
4.8 workflows/ — 顶层科研流程

```
from sqc.workflows.predistortion_validation import PredistortionValidationWorkflow
wf = PredistortionValidationWorkflow(qubit=q, target_waveform=target,
true_distortion=dist)
results = wf.run()
print(results["metrics"]["improvement_factor"])

from sqc.workflows.z_crosstalk import ZCrosstalkWorkflow
wf = ZCrosstalkWorkflow(chip=chip, flux_pulse_on_A=pulse, true_transfer_matrix=tm)
results = wf.run()
```

5. 数据流

5.1 完整数据流图



5.2 标定辅助流

```
Calibration.calibrate(qubit)
|
```

```
└─ QubitFrequencyCalibration → CalibrationTable( $f_{01}$ )
└─ FluxResponseCalibration → CalibrationTable( $\phi$  vs  $h$ )
└─ TransferFunctionCalibration → CalibrationTable( $H(\omega)$ )
```

6. 快速开始

6.1 环境配置

```
pip install -r requirements.txt
python -c "import qutip, numpy, scipy, matplotlib; print('ok')"
```

6.2 基础用法

```
import numpy as np
from sqc.devices.transmon import TransmonQubit, QubitSpec
from sqc.control.flux_signal import FluxSignal
from sqc.experiments.ramsey import RamseyExperiment

# 1. 创建 qubit
q = TransmonQubit(EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=10000, T2=8000)

# 2. 创建磁通信号
phi = FluxSignal(type=2, t_list=np.linspace(0, 250, 500),
                 amplitude=0.001, frequency=0.01)

# 3. 运行 Ramsey 实验
exp = RamseyExperiment(qubit=q, flux_signal=phi)
result = exp.run()
print(result.data["p_e"]) # Ramsey 干涉条纹
```

6.3 使用新架构（推荐）

```
from sqc.devices.transmon import QubitSpec
from sqc.simulation.hamiltonian import HamiltonianBuilder

spec = QubitSpec("Q0", EC=2*np.pi*0.2, EJ=2*np.pi*15, T1=1e4, T2=8e3)
H_list, t_list = HamiltonianBuilder.build(spec, flux_signal=phi)
# spec 未被修改 ✓
```

6.4 向后兼容

```
# 旧 import 仍然可用（通过 src/ 不变 + src_mirror/ 重导出）
from src.qubit import TransmonQubit # 原始 src/（未被修改）
```

```
from src_mirror.qubit import TransmonQubit # facade → sqc/

# 两者均可工作，但新代码建议直接用 sqc/
from sqc.devices.transmon import TransmonQubit
```

6.5 运行测试

```
pytest tests/unit -v          # 快速单元测试
pytest tests/regression -v    # 物理回归测试
pytest tests/ -v              # 全部测试
```

6.6 启动 Web Demo

```
python web_demo.py           # 旧版 Gradio demo (兼容)
python web_demo_v2.py        # 新版可视化 demo
```

7. 扩展指南

7.1 添加新的实验协议

1. 在 `sqc/experiments/` 创建新文件（如 `my_protocol.py`）
2. 继承 `Experiment ABC`，实现 `build_sequence()` 和 `run()`：

```
from sqc.experiments.base import Experiment

@dataclass
class MyProtocolExperiment(Experiment):
    qubit: TransmonQubit
    param1: float = 1.0

    def build_sequence(self):
        return create_my_pulse_sequence(...)

    def run(self) -> ExperimentResult:
        self.qubit.qubit_in_mag(self.flux_signal)
        # ... mesolve ...
        return ExperimentResult(data={...}, axes={...}, metadata={...}, config=
{...})
```

3. 在 `src_mirror/protocal.py` 添加新的 case 转发
4. 添加测试到 `tests/integration/`

7.2 添加新的重建算法

1. 在 `sqc/reconstruction/` 创建新文件
2. 继承 `Reconstruction` ABC:

```
from sqc.reconstruction.base import Reconstruction

@dataclass
class MyReconstruction(Reconstruction):
    param: float = 1.0

    def reconstruct(self, measurement, kernel=None, calibration=None, **kwargs):
        # 算法逻辑
        return reconstructed_flux_signal
```

3. 在 `src_mirror/analysis.py` 的 `Analysis` 类中添加 facade 方法

7.3 添加新的失真模型

1. 在 `sqc/hardware/distortion.py` 添加新类
2. 继承 `DistortionModel` ABC，实现四个抽象方法
3. 在 `src_mirror/distortion.py` 添加 re-export

7.4 扩展规范

- **命名**: 类名用 `PascalCase`，文件名用 `snake_case`
- **类型注解**: 所有公共方法必须有类型注解 (`from __future__ import annotations`)
- **docstring**: 英文，numpy 风格，包含 `Parameters / Returns`
- **不可变性偏好**: 新数据结构优先使用 `@dataclass(frozen=True)`
- **facade 保留**: `src_mirror/` 应始终提供向后兼容的重导出
- **测试要求**: 每个新模块 ≥ 3 个单元测试，涉及物理计算的必须有回归测试

7.5 添加新 qubit 参数必须通过 sanity check

引入新参数前验证其落在 Gao 2021 推荐范围内 (§2.3)，偏离范围须在 commit message 中说明物理动机。

8. API 参考

8.1 核心类索引

类	模块	用途
<code>QubitSpec</code>	<code>sqc.devices.transmon</code>	不可变 qubit 参数
<code>TransmonQubit</code>	<code>sqc.devices.transmon</code>	向后兼容 qubit 类
<code>Resonator</code>	<code>sqc.devices.resonator</code>	多模谐振腔
<code>ChipTopology</code>	<code>sqc.devices.chip</code>	多 qubit 芯片拓扑
<code>Waveform</code>	<code>sqc.control.waveform</code>	通用时域波形

类	模块	用途
FluxSignal	sqc.control.flux_signal	磁通信号
Pulse	sqc.control.pulse	微波控制脉冲
CompositePulse	sqc.control.pulse	复合脉冲序列
ControlLine	sqc.hardware.control_line	物理控制线
SingleExponentialDistortion	sqc.hardware.distortion	单指数失真
TransferMatrix	sqc.hardware.transfer_matrix	Z 线串扰矩阵
HamiltonianBuilder	sqc.simulation.hamiltonian	无副作用 H 构造
MesolveRunner	sqc.simulation.runner	mesolve 执行器
ExperimentResult	sqc.simulation.result	实验结果容器
Experiment	sqc.experiments.base	实验 ABC
RamseyExperiment	sqc.experiments.ramsey	Ramsey 协议
KernelEstimator	sqc.reconstruction.kernel	控制核估计
WienerReconstruction	sqc.reconstruction.wiener	Wiener 反卷积
LMReconstruction	sqc.reconstruction.numerical_inverse	LM 数值反演
CalibrationTable	sqc.calibration.base	标定结果表
PredistortionDesigner	sqc.calibration.predistortion	预失真设计器

8.2 关键函数签名

```

# HamiltonianBuilder
HamiltonianBuilder.build(qubit, flux_signal, pulse, frame, omega_d) -> tuple[list,
np.ndarray]

# KernelEstimator
KernelEstimator.estimate(pulse, qubit) -> tuple[np.ndarray, np.ndarray] #
(t_samples, kernel)

# WienerReconstruction
WienerReconstruction(lambda_reg=1.0).reconstruct(measurement, kernel,
calibration=None, dt=None) -> FluxSignal

# LMReconstruction
LMReconstruction(qubit, control_pulse, basis_type="fourier", n_basis=100,
...).reconstruct(measurement) -> tuple[FluxSignal, dict]

# ControlLine
ControlLine.apply(awg_waveform: Waveform) -> Waveform
ControlLine.predistort(target_waveform: Waveform, designer: PredistortionDesigner)
-> Waveform

```

```
# TransferMatrix
TransferMatrix.apply(source_voltages: dict[str, Waveform]) -> dict[str,
FluxSignal]
```

9. 测试与回归

9.1 测试套件

套件	位置	marker	用途
单元测试	tests/unit/	@pytest.mark.unit	纯函数/类单元验证
集成测试	tests/integration/	@pytest.mark.integration	跨模块管道验证
回归测试	tests/regression/	@pytest.mark.regression	物理结果锚定
等价性测试	tests/equivalence/	—	旧 src vs 新 sqc 数值等价

9.2 物理回归机制

- 1. **baseline 生成**: `python -m tests.regression.generate_baselines`
- 2. **回归验证**: `pytest tests/regression -m regression`
- 3. **容限**: `rtol=1e-6, atol=1e-9` (不可放宽)
- 4. **baseline 更新**: 仅当有意改变物理行为时重生成, commit message 必须显式说明

9.3 baseline 清单

文件	内容
qubit_static.pkl	f_{01}, α, κ 静态属性
ramsey_default.pkl	Protocal case 1 (Ramsey)
diff_echo_default.pkl	Protocal case 2 (差分回波)
transient_default.pkl	Protocal case 4 (瞬态传感)
lm_default.pkl	LM 数值反演
predistortion_default.pkl	预失真验证
z_crosstalk_default.pkl	Z 串扰提取

10. 全局配置 (Global Configuration)

10.1 设计理念

`sqc/config.py` 提供统一的全局配置体系。参数从硬件层向上推导, 而不是在不同文件中各自硬编码。

```
AWG sample_rate (hardware)
→ dt = 1 / sample_rate (全局时间量子)
→ t_rabi = arange(0, 10, dt) (标准  $\pi$  脉冲窗口)
→ t_global = arange(-50, 400, dt) (仿真时间窗)
→ t_signal = arange(0, 250, dt) (磁通信号默认时间轴)
```

所有时间轴使用 `np.arange(start, stop, dt)` 而非 `np.linspace(start, stop, N)`，以确保：

- 每个时间点落在整数 AWG 采样边界上
- `dt` 全局唯一，从 `AWGConfig.sample_rate` 推导
- 脉冲面积计算不依赖不整齐的步长 (`amplitude = pi / duration`)

10.2 配置层次

Section	Dataclass	关键字段
Hardware	<code>AWGConfig</code>	<code>sample_rate: 2.0 GSa/s → dt: 0.5 ns</code>
	<code>ControlLineDefaults</code>	<code>impedance: 50 Ω, attenuation_db: 20 dB</code>
Devices	<code>TransmonDefaults</code>	<code>EC: 0.2, EJ: 15.0, T1: 10k ns, n_levels: 3</code>
Control	<code>PulseConfig</code>	<code>t_rabi_duration: 10 ns, t_global_start/end, dt (injected)</code>
Simulation	<code>SimulationConfig</code>	<code>atol: 1e-8, rtol: 1e-6</code>
Reconstruction	<code>ReconstructionConfig</code>	<code>lambda_reg: 1.0, stim_amplitude: 0.0215, lm_n_basis: 100</code>

10.3 使用方式

```
from sqc.config import CONFIG, reconfigure

# 获取全局默认值
dt = CONFIG.awg.dt # 0.5 ns
t_rabi = CONFIG.pulse.t_rabi # arange(0, 10, dt)
t_cons = CONFIG.transmon # TransmonDefaults
q = TransmonQubit(**t_cons.to_dict())

# 创建自定义配置（全局单例不变）
cfg2 = reconfigure(sample_rate=4.0, t_rabi_duration=20)
exp = RamseyExperiment(qubit=q, t_rabi=cfg2.pulse.t_rabi)
```

10.4 已适配模块

所有时间轴从 `CONFIG.pulse` 获取默认值：

- `sqc/experiments/`: Ramsey, Echo, Rabi, Transient, Cryoscope

- `sqc/calibration/`: QubitFrequency, FluxResponse
- `sqc/control/`: sequence.py (free-evolution gaps), flux_signal.py (default t_signal)
- `sqc/hardware/`: readout.py (t_rabi)
- `sqc/workflows/`: z_crosstalk.py (t_rabi)

11. 设计原则与约定

11.1 核心原则

1. **物理结果不变**：重构不改变任何物理算法的等价行为
2. **device/qubit 只描述物理参数**：不存储实验状态
3. **HamiltonianBuilder 无副作用**：输入不被修改
4. **experiment 是组合**：不是巨函数
5. **reconstruction 只消费数据**：不做仿真（LM 通过依赖注入）
6. **hardware 显式建模**：反映真实 cQED 栈
7. **永久镜像**：`src/` 不删除，旧代码无限期可工作
8. **不引入新依赖**：只用 numpy/scipy/qutip/matplotlib/dataclasses
9. **全局配置中心化**：所有时间网格从 `CONFIG.awg.dt` 推导，不使用 `np.linspace`

10.2 命名与拼写

- `Protocal`（故意错拼）在 `src/` 和 `src_mirror/` 中**永远保留**
- `sliding_measrement`（故意错拼）在 `facade` 中保留
- `sqc/` 中新代码使用正确拼写：`Protocol`, `Experiment`, `Calibration`

10.3 单位约定

物理量	单位
频率/能量	rad·GHz（含 2π ）
时间	ns
磁通	Φ_0
$\hbar$	1（自然单位）

10.4 与 Gao 2021 的差异

项	论文	本项目	理由
算符表示	ladder operator	QuTiP destroy/num	数值等价
噪声模型	多渠道 T_1 , T_2 , T_φ	Lindblad c_ops	标准 QuTiP
Readout	dispersive cavity + IQ	Ramsey-based IQ	本项目核心是磁通传感
校准循环	完整 Fig.9 依赖图	简化为三类	聚焦三大主线

参考文献

- **[Gao 2021]** Y. Y. Gao, M. A. Rol, S. Touzard, and C. Wang, "Practical Guide for Building Superconducting Quantum Devices", *PRX Quantum* **2**, 040202 (2021)
 - **[Koch 2007]** J. Koch et al., "Charge-insensitive qubit design derived from the Cooper pair box", *Phys. Rev. A* **76**, 042319 (2007)
 - **[Motzoi 2009]** F. Motzoi et al., "Simple Pulses for Elimination of Leakage in Weakly Nonlinear Qubits", *Phys. Rev. Lett.* **103**, 110501 (2009)
-

文档结束。下一步：阅读 [idea/refactor/_refactor_plan.md](#) 了解完整设计背景。